



Sharif University of Technology

Department of Electrical Engineering

Course:

ASIC/FPGA

Report:

ASIC/FPGA Project - Group 8

Weekly Report III

Erik Hartouni

401102778

Ali Behrad

401101321

Submitted by:

Mohammadhossein Sabzalian

402101906

Sina Hashemi

402102668

Instructor:

Dr. Baharvand

Department of Electrical
Engineering

Term: Spring 2026

Project Management & Synthesis Hand-off

Week 3 followed the guideline's **Synthesis & Timing Closure** phase. With functional verification closed out in Week 2 (35/35 assertions passing against the RTL model), the split for this week put **Sina Hashemi** and **Erik Hartouni** (roles A, C) on logic synthesis, timing-constraint definition, and area/timing closure, while **Mohammadhossein Sabzalian** and **Ali Behrad** (roles B, D) brought up the post-synthesis gate-level simulation (GLS) flow and compared it against the Week 2 functional baseline. As Week 3 integration lead, this report was compiled by **Erik Hartouni** (role C).

Others:

- **Repository update:** The synthesis TCL scripts, the exported gate-level netlist (`otp_controller_postsyn.v`), the back-annotated SDF (`otp_controller_postsyn.sdf`), and the Genus terminal logs were committed under `synthesis/`, alongside the Week 1-2 RTL and testbench tree.
- **Testbench compatibility:** No major changes to `tb_otp_controller.v` were required to add GLS support; we simply added a few lines to read from the post-synthesis netlist and the corresponding `.sdf` file whenever `'POSTSYN_SIM` is defined (Section).

Week 3 Plan vs. Actual

Planned Activity (Weekly Schedule)	Responsible	Status
Run synthesis (area report, timing report)	A, C	Done
Analyze synthesis results, address warnings	A, C	Done (Sec.)
Optimize RTL if timing/area not met	A, B	Not needed (timing met on first pass)
Re-run synthesis after optimization	A, C	No Optimization needed
Run post-synthesis gate-level simulation	B, D	Done (Sec.)
Compare pre-syn and post-syn results	B, D	Done – 39/39 GLS checks match RTL behaviour
Integration Lead (Week 3) reports progress	C	Done

Table 1: Week 3 deliverables against the schedule defined in the project guideline.

Post-Synthesis Simulation Flow

Verification Baseline

Before attempting GLS, the Week 2 functional regression (`func_sim.tcl`) was re-run unmodified against the RTL to confirm the simulation environment itself – compile options, include paths, and the self-checking testbench – was still stable. This baseline check is what lets any GLS failure later be attributed to a [synthesis mismatch or timing violation](#), rather than to an environment or testbench regression, exactly as required by the Week 3 success criterion (“*Gate sim matches RTL sim*”).

Enabling POSTSYN_SIM

Rather than duplicating the testbench for gate-level use, `tb_otp_controller.v` was written in Week 2 with the structural netlist, the TSMC cell library, and the SDF back-annotation call gated behind a single ``ifdef POSTSYN_SIM` switch, so the same file drives both `func_sim.tcl` (RTL) and `post_syn_sim.tcl` (gate-level) depending on whether `+define+POSTSYN_SIM` is passed to the simulator invocation.

Listing 1: Compile-Time Switch Between RTL and Structural DUT (Testbench)

```
1 `ifdef POSTSYN_SIM
2     `include "/home/cad/TECH.D/TSMC180/Verilog/tsmc18.v"
3     `include "../synthesis/synout/otp_controller_postsyn.v"
4 `endif
5 ...
6 initial begin
7     `ifdef POSTSYN_SIM
8         $sdf_annotate("../synthesis/synout/otp_controller_postsyn.sdf",
9             u_dut, , , "MAXIMUM");
10        $display("[POSTSYN] SDF annotation requested at t=%0t ...", $time
11            );
12    `endif
13    $readmemh("../rtl_sources/otp_image.mem", golden_img);
14    ...
```

- **Same Test Sequence, Different DUT:** Every stimulus, scoreboard check, and golden-image comparison written for Week 2 runs unchanged; only the module bound to `u_dut` changes, from the behavioral `otp_controller.v` to the Genus-mapped structural netlist.
- **MAXIMUM Delay Mode:** `$sdf_annotate` is called with the "MAXIMUM" argument, so the simulation is back-annotated with worst-case (slow-corner) TSMC gate delays – the pessimistic condition relevant to timing closure, not the typical or best-case corner.
- **The OTP Array Stays Behavioral:** `u_rom` (`otp_sim_rom.v`) is *not* part of the synthesized netlist and is left as a behavioral `$readmemh`-loaded model in both RTL and GLS runs. This is intentional: DRS Section 2 explicitly states the OTP array “is modelled as a synchronous ROM for RTL simulation; the synthesis target can be

a register file or a real OTP macro,” i.e. the 256-bit array is an *abstract behavioral placeholder* in this exercise, and a real OTP block is a foundry-specific non-volatile macro with its own physical implementation that is out of scope for Genus synthesis against a standard-cell library. Only `otp_controller` (the FSM, counters, CRC engine, and output registers) was synthesized.

SDF Back-Annotation & Hierarchy

Loading the structural netlist together with the TSMC cell models and the back-annotated SDF is the first gatekeeper of physical verification: if the SDF’s timing paths do not structurally match the netlist, Xcelium reports annotation errors before a single test vector is run.

```

Annotation completed with 0 Errors and 476 Warnings
SDF statistics:
  No. of Pathdelays = 4979      No. of Disabled Pathdelays = 0      Annotated
= 100.00% (4979/4979)
  No. of Tchecks   = 2984      No. of Disabled Tchecks   = 0      Annotated
= 84.05% (2508/2984)

Percentage(A/(T-D))
Path Delays      4979      0      4979
$width           1428      0      1428
$setuphold       1556      0      1080
69.41
Building instance overlay tables: ..... Done
Generating native compiled code:
  worklib.otp_controller:v <0x54bb7aff>
    streams: 0, words: 0
  worklib.tb_otp_controller:v <0x4e942072>
    streams: 20, words: 44818
  worklib.otp_sim_rom:v <0x27b1e5db>
    streams: 3, words: 1072
Building instance specific data structures.
Loading native compiled code: ..... Done
Design hierarchy summary:
      Instances  Unique
Modules:      1840    46
UDPs:         511     3
Primitives:   4725     8
Timing outputs: 1839    20
Registers:    540     72
Scalar wires: 3822     -
Always blocks: 4        4
Initial blocks: 5        5
Cont. assignments: 0      3
Pseudo assignments: 2     2
Timing checks: 4540   1040
Interconnect: 5452     -
Simulation timescale: 1ps
Writing initial simulation snapshot: worklib.tb_otp_controller:v
xmsim: *W,NOMTDGUI: Multi-Threaded Dumping is disabled for interactive debug mode.

```

Figure 1: SDF annotation summary and simulation design-hierarchy snapshot for the gate-level testbench.

SDF Annotation Result Annotation completed with **0 errors** and **476 warnings**. All **4,979/4,979** (100%) gate-to-gate path delays and all **1,428/1,428** (100%) pulse-width timing checks were successfully annotated; only the \$setphold checks were partially annotated, at **1,080/1,556 (69.41%)**. Zero errors confirms the structural netlist and the SDF file are consistent – every delay path in the SDF found a matching net in the design. These 476 warnings stem from TSMC library modules/timing arcs that our design does not actually exercise; see the full disposition in Section .

The design-hierarchy summary in the same log is a useful cross-check on the synthesis results reported in Section : the simulation snapshot reports **1,840 module instances / 46 unique modules** and **540 registers / 72 unique**, both slightly larger than the **1,836-cell / 476-flip-flop otp_controller** netlist alone (Section), because the simulation hierarchy also includes the testbench, the behavioral `otp_sim_rom`, and the top-level wrapper on top of the synthesized DUT.

GLS Test Results

The full six-test regression from Week 2 (DRS §7.1 #1–#5 plus the optional CRC-error check) was re-run against the back-annotated gate-level netlist, at the same 131.072 kHz clock used throughout the project.

Gate-Level Regression Summary **39 of 39 checks passed, 0 failed**, and the run concluded with **RESULT: ALL TESTS PASSED**. Every category from the RTL regression – reset behaviour, continuous busy during load, correct field extraction (13/13 registers), patch precedence, invalid-patch rejection, and CRC error detection – held under back-annotated gate delays. The check count was already 39/39 at the Week 2 RTL baseline (the watchdog assertion `wait_for_done` already fires, and passes, ahead of Tests 3–6), so no new checks were introduced when moving to gate-level simulation – the same 39-check regression simply passed again under back-annotated timing.

This is the Week 3 success criterion (“*Gate sim matches RTL sim*”) satisfied directly: functional behaviour is unchanged between the behavioral and structural DUT, so any differences observed are confined to timing, area, and power – covered next.

Synthesis Flow: Scripts & Clock Constraints

Synthesis Workspace & Scripts

Rather than a single monolithic script, the synthesis flow was partitioned into dedicated TCL sub-scripts, following standard Cadence Genus practice: one script per library setup, one per timing constraints, and one for the compilation flow itself. This separation lets the target library or clock constraint be swapped (e.g. nominal vs. worst-case corner) without touching the compilation logic.

- **lib_scr.tcl** – specifies the target TSMC standard-cell library database files and sets operating conditions, wire-load models, and link paths, so Genus knows the timing, area, and power characteristics of every gate it can substitute for the behavioral RTL.
- **otp_controller_clock_const.tcl** – sets the clk port frequency to the DRS-mandated **131.072 kHz** system requirement.
- **syn_scr.tcl** – the master script: sources **lib_scr.tcl**, reads the **otp_controller** RTL sources, and elaborates the top-level module into a generic technology-independent gate representation before mapping and optimization.

```
[grp8_dsd@bahar synthesis]$ ls
constraints.g  genus.log3      otp_controller.mmmc.tcl
fv            genus.log4      otp_controller.mode
genus.cmd     otp_controller.default_emulate_constraint_mode.sdc  otp_controller.mode_attributes.taf.gz
genus.cmd1    otp_controller.flowkit_settings.tcl                otp_controller.ndr.tcl
genus.cmd2    otp_controller.g                                otp_controller.preserve.tcl
genus.cmd3    otp_controller.genus_setup.tcl                     otp_controller.v
genus.cmd4    otp_controller.invs_init.tcl                       otp_controller.wnm_attrs.tcl
genus.log     otp_controller.invs_setup.tcl                      script
genus.log1    otp_controller.metrics.json                        synout
genus.log2    otp_controller.min_layer
```

Figure 2: Synthesis working directory after a completed Genus run, showing the partitioned TCL scripts and generated synout/script outputs.

Synthesis Execution

The synthesis run was launched interactively with **genus** and driven entirely by **syn_scr.tcl**.

```
[grp8_dsd@bahar synthesis]$ genus
TMPDIR is being set to /tmp/genus_temp_88587_bahar_grp8_dsd_9MeIGA
Cadence Genus(TM) Synthesis Solution.
Version: 21.10-p002_1, built Thu Aug 19 22:43:13 PDT 2021
Checking out license: Genus_Synthesis
Finished loading tool scripts (12 seconds elapsed).
@genus:root: 1>
```

Figure 3: Cadence Genus (v21.10-p002_1) startup: license checkout and tool-script loading before sourcing **syn_scr.tcl**.

After generic mapping (SYNTH-5) and incremental gate restructuring (SYNTH-7) against the TSMC library, Genus reports the clock it synthesized against:

```

Stage : incr_opt
SYNTH-5 | Info | Done mapping.
SYNTH-7 | Info | Incrementally optimizing.
...
Clock Description
-----
Clock   Period   Rise   Fall   Clock   Source   No of
Name                                Domain  Pin/Port Registers
-----
clk     7629390.0  0.0   3814695.0 sysclk  clk      476

```

Figure 4: Incremental-optimization stage completion and the resulting clock description for the `clk` domain.

Clock Verification The reported clock period is exactly **7,629,390 ps = 7.62939 μs**, which corresponds to

$$F = \frac{1}{7.62939 \times 10^{-6} \text{ s}} = 131.072 \text{ kHz},$$

an exact match to the DRS §2.3 clock requirement, driven from the `sysclk` source pin. The domain contains **476 sequential registers**, consistent with the flip-flop count reported in the area breakdown of Section .

Timing, Area & Power Closure

Timing Closure

Genus reports the single worst-case setup path in the design ([Path 1](#)):

Path 1: MET (7626537 ps) Setup Check with Pin ratio_p1_reg[19]/CK->D
Startpoint: (R) state_reg[0]/CK
Clock: (R) clk
Endpoint: (R) ratio_p1_reg[19]/D
Clock: (R) clk
Capture Edge:+ 7629390
Required Time:= 7629264
Data Path:= 2727
Slack:= 7626537

Figure 5: Worst-case setup timing path: state_reg[0]/CK to ratio_p1_reg[19]/D.

- **Critical path:** starts at the FSM state register's clock pin (state_reg[0]/CK) and ends at bit 19 of the 20-bit calibration ratio register ratio_p1_reg[19]/D – i.e. the longest combinational fan-out from the FSM decode logic into the widest output field (RATIO_P1, DRS Table 3.1).
- **Data path delay:** only **2,727 ps (2.727 ns)** to propagate through the combinational steering logic (INVX1, OR2X4, AND2X2, NOR2BX1, AOI22X1 cells) between the two registers.
- **Slack:** **+7,626,537 ps** against a 7,629,390 ps period – **Path 1: MET**. Because the 131.072 kHz target clock period is nearly four orders of magnitude longer than any nanosecond-scale gate delay in an 180 nm library, timing closure at this frequency is not a meaningful constraint; the design closes with margin to spare on the very first synthesis pass, which is why the “re-run synthesis after RTL optimization” step in Table 1 was not needed.

Area Breakdown

Instance	Module	Cell Count	Cell Area	Total Area

otp_controller		1836	51831.965	51831.965
u_crc	crc16_ccitt	6	103.118	103.118

Type	Instances	Area	Area %	

sequential	476	33629.905	64.9	
inverter	51	339.293	0.7	
logic	1309	17862.768	34.5	

total	1836	51831.965	100.0	

Figure 6: Cell count and area by instance and by cell type for the synthesized otp_controller.

Cell Type	Instances	Area (μm^2)	Area %
Sequential (flip-flops)	476	33,629.905	64.9%
Combinational logic	1,309	17,862.768	34.5%
Inverters	51	339.293	0.7%
Total	1,836	51,831.965	100.0%

Table 2: Area ratio breakdown of the synthesized `otp_controller` (includes the 6-cell, $103.118\mu\text{m}^2$ `u_crc` sub-instance).

Area Ratio Analysis Sequential cells alone account for **64.9%** of total area despite being only 26% of instance count (476 of 1,836) – each flip-flop is simply a larger primitive than a 2-input gate. This ratio is exactly what is expected of an **OTP controller**: DRS §4.2 defines more than a dozen static output registers (`RO_TRIM_CODE`, `TC1_COEFF`, `TC2_COEFF`, `AGING_BASE`, five `RATIO_Px` registers, etc.) that must *hold their value indefinitely* after done, so most of the silicon budget is storage, not decision logic. The `u_crc` (CRC-16/CCITT) sub-block is comparatively tiny at $103.118\mu\text{m}^2$ / 6 cells – under 0.2% of total area – confirming the CRC check is essentially free relative to the register file it protects.

Power Estimation

Category	Leakage	Internal	Switching	Total	Row%
register	1.65826e-07	5.14986e-06	1.56738e-07	5.47242e-06	82.51%
logic	4.28900e-08	3.23326e-07	1.73490e-07	5.39705e-07	8.14%
clock	0.00000e+00	0.00000e+00	6.20405e-07	6.20405e-07	9.35%
Subtotal	2.08716e-07	5.47318e-06	9.50632e-07	6.63253e-06	100.00%
Percentage	3.15%	82.52%	14.33%	100.00%	

Figure 7: Vectorless static power estimate by category (leakage, internal, switching).

- **Total power: 6.633 μW** (leakage $0.209\mu\text{W}$ / 3.15%, internal $5.473\mu\text{W}$ / 82.52%, switching $0.951\mu\text{W}$ / 14.33%), dominated overwhelmingly by internal switching, not leakage or net-wire charging.
- **Register power dominance:** the `register` category alone contributes **82.51%** of total power ($5.472\mu\text{W}$ of $6.633\mu\text{W}$), directly mirroring its 64.9% share of area – because the 131.072 kHz clock is so slow, gate and interconnect switching activity (`logic`: 8.14%, `clock`: 9.35%) is negligible, and the dominant power cost is simply toggling and holding the 476 output/state flip-flops.
- **Consistency check:** both the area and power breakdowns independently point to the same conclusion – this design’s cost is register-dominated, which is the expected signature of a configuration-storage block rather than a compute-heavy datapath.

Design Export & Warnings

Netlist & SDF Export

```
@file(syn_scr.tcl) 117: write_hdl > ./synout/otp_controller_postsyn.v
@file(syn_scr.tcl) 118: write_sdf -version 3.0 ... > ./synout/otp_controller_postsyn.sdf
@file(syn_scr.tcl) 122: write_script > ./constraints.g
@file(syn_scr.tcl) 124: write_design -basename otp_controller ... -innovus ...
Exporting design data for 'otp_controller' to ./otp_controller...
Finished exporting design data for 'otp_controller'
```

Figure 8: Final `syn_scr.tcl` export commands: structural netlist, SDF, constraints, and Innovus design database.

The last stage of `syn_scr.tcl` writes exactly the artifacts the GLS flow of Section depends on: the gate-level netlist (`write_hdl` → `otp_controller_postsyn.v`), the back-annotated timing (`write_sdf` → `otp_controller_postsyn.sdf`), a re-runnable constraints snapshot (`write_script` → `constraints.g`), and a full design database for downstream place-and-route in Innovus (`write_design ... -innovus`). The clean “*Finished exporting design data*” message confirms the hand-off package is complete.

Synthesis & Simulation Warnings

Two distinct warning sources appeared this week, and both were traced to the TSMC library models rather than to the RTL or the testbench:

- **Simulation-side (476 `$setuphold` warnings, Fig. 1):** these are emitted by Xcelium while annotating the SDF against the TSMC cell timing models, not by `otp_controller` itself – they reflect setup/hold check arcs defined inside the vendor’s own `tsmc18.v` primitive models that the annotator could not fully resolve (69.41% `$setuphold` annotation vs. 100% for path delays and width checks). These are timing-check arcs belonging to TSMC library primitives/modules that our design does not actually use, not defects in `otp_controller`. Because zero SDF annotation *errors* were reported and all 39 GLS checks still passed, these warnings did not affect functional or timing correctness.
- **Synthesis-side (Genus log):** the remaining warnings surfaced during elaboration and mapping are library-characterization notices tied to the TSMC 180 nm standard-cell database (`lib_scr.tcl`) – e.g. unused/undriven pin conditions and corner coverage notes intrinsic to the vendor `.lib` files – rather than structural or functional issues with the synthesized RTL. No SYNTH-stage error was reported, and the design elaborated, mapped, and closed timing cleanly on the first pass.

Why the OTP Array Was Not Synthesized: As established in Section , the 256-bit OTP array (`otp_sim_rom.v`) was deliberately excluded from the Genus run. DRS §2.1 models it only as a *synchronous ROM abstraction* for RTL-level simulation; a real OTP macro is a foundry-specific non-volatile memory IP block with its

own bit-cell technology, programming circuitry, and timing, none of which can be described in synthesizable standard-cell RTL. Synthesizing only `otp_controller` – and keeping `otp_sim_rom` as a behavioral, `$readmemh`-loaded stand-in on both sides of the GLS comparison – keeps the gate-level results meaningful (real gates, real delays for the *controller* logic) without fabricating a nonexistent gate-level model for hardware that will, in real silicon, be a licensed analog/mixed-signal macro dropped in at the physical-design stage.

Deliverables & Outlook

All Week 3 deliverables from the schedule were met: a synthesized gate-level netlist (1,836 cells, 51,831.965 μm^2) meeting timing with 7.626 μs of slack at 131.072 kHz; a passing gate-level simulation (39/39 checks, back-annotated with **MAXIMUM** SDF delays); an area/power closure analysis showing the design is register-dominated as expected for a configuration-storage block; and this progress report from the Week 3 integration lead. With both pre- and post-synthesis behaviour verified equivalent and no timing or area risk identified, Week 4 will shift to cross-group integration with Group 7 (redundant OTP Controller implementation), top-level simulation with neighboring modules, and assembly of the final report and code package per the Week 4 deliverables list.